

Event Management System API Documentation

Overview

The Event Management System API provides endpoints for user authentication, category management, vendor discovery and management, package management, and order processing.

This API appears to be organized into the following modules:

- Authentication
- Category
- Vendor
- Package
- Order

The collection currently contains multiple local-style route patterns, which suggests the API may be under active development or assembled from separate modules.

Base URL

The collection uses multiple route roots instead of one consistent base URL. Examples include:

- `/3DOS/Event Managment/Team3/Event/routes/Auth.php`
- `/3DOS/Event Managment/Team3/Event/routes/Vendor.php`
- `/3DOS /Event/Routes/categories.php`
- `/3DOS /Event/Routes/Vendor.php`
- `/3DOS /Routes/Package.php`
- `/3DOS /Order/Route/OrderRoute.php`

Authentication

Authentication in the collection is partially configured. Observed patterns:

- Some endpoints use Bearer token authentication
- Some endpoints do not explicitly define auth in the saved request
- Profile/admin-style endpoints appear protected
- Login and signup endpoints appear public

Auth Header

For protected endpoints, use:

Authorization: Bearer <token>

Common Response Format

The collection does not include saved examples, so the following is a recommended standard format for documentation.

Success Response

```
{
  "success": true,
  "message": "Request completed successfully",
  "data": {}
}
```

Error Response

```
{
  "success": false,
  "message": "Validation failed",
  "errors": {
    "field": "Error details"
  }
}
```

Common HTTP Status Codes

Status Code	Meaning
200	OK
201	Created
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
409	Conflict
422	Validation Error
500	Internal Server Error

Pagination

Some endpoints support pagination using query parameters such as:

- `page`
- `limit`

Example:

```
GET /orders?page=1&limit=5
```

Recommended paginated response format:

```
{
  "success": true,
  "data": [],
  "pagination": {
    "page": 1,
    "limit": 10,
    "total": 100,
    "pages": 10
  }
}
```

Search and Filtering

The Vendor module supports filtering and search through query parameters such as:

- `search`
- `categoryId`
- `location`
- `day`
- `action`

Example:

```
GET /Vendor.php?categoryId=2&location=cairo
```

Security Notes

Recommended security practices for this API:

- Use HTTPS in all non-local environments
- Store tokens securely
- Validate all user input server-side
- Enforce role-based authorization for admin/vendor endpoints
- Sanitize search/filter input
- Protect authentication endpoints against brute-force attacks
- Never expose sensitive data in error messages

Architecture Notes

Based on the collection, the system appears to include these core domains:

- Users / Authentication
- Vendors
- Categories
- Packages
- Orders
- Wallet / vendor earnings

The routing style suggests a PHP-based backend with module-specific route files.

Database Entities

Based on request structure, the likely entities include:

- User
- Vendor
- Category
- Package
- Order
- Wallet
- Review

Possible relationships:

- A vendor belongs to a category
- A vendor can create many packages
- A user can create many orders
- A package can receive reviews
- A vendor may have wallet or balance records

Authentication Module

1. User Signup

Method: POST
Endpoint: /3DOS/Event_Managment/Team3/Event/routes/Auth.php/signup
Description: Register a new user account.

Authentication

No authentication required.

Headers

Content-Type: application/json

Request Body

```
{
  "Name": "John Doe",
  "Email": "john@example.com",
  "Password": "your password",
```

```
"PhoneNumber": "01234567890"
}
```

Success Response

```
{
  "success": true,
  "message": "User registered successfully",
  "data": {
    "userId": 1
  }
}
```

Error Responses

- 400 Bad Request
- 409 Conflict
- 422 Validation Error

2. User Login

Method: POST

Endpoint: /3DOS/Event Managment/Team3/Event/routes/Auth.php/login

Description: Authenticate a user and return an access token or session.

Authentication

No authentication required.

Headers

Content-Type: application/json

Request Body

```
{
  "Email": "john@example.com",
  "Password": "your_password"
}
```

Success Response

```
{
  "success": true,
  "message": "Login successful",
  "data": {
    "token": "jwt_token_here",
    "user": {}
  }
}
```

Error Responses

- 400 Bad Request
- 401 Unauthorized

3. Get Current User

Method: GET

Endpoint: /3DOS/Event_Managment/Team3/Event/routes/Auth.php/me

Description: Retrieve the currently authenticated user profile.

Authentication

Bearer token required.

Headers

Authorization: Bearer <token>

Query Parameters

None.

Request Body

No body is expected for a GET request.

Success Response

```
{
  "success": true,
  "data": {
    "id": 1,
    "name": "John Doe",
    "email": "john@example.com"
  }
}
```

Error Responses

- 401 Unauthorized
- 404 Not Found

4. Vendor Login

Method: POST

Endpoint: /3DOS/Event_Managment/Team3/Event/routes/Auth.php/vendor/login

Description: Authenticate a vendor account.

Authentication

No authentication required.

Headers

Content-Type: application/json

Request Body

```
{
  "Email": "vendor@example.com",
  "Password": "your_password"
}
```

Success Response

```
{
  "success": true,
  "message": "Vendor login successful",
  "data": {
    "token": "jwt_token_here",
    "vendor": {}
  }
}
```

Error Responses

- 400 Bad Request
- 401 Unauthorized

5. Register Vendor

Method: GET

Endpoint: Not configured

Description: This request appears incomplete in the collection and should be corrected before publishing final documentation.

Notes

- saved method is likely incorrect
- URL is missing
- request should be reviewed before release

Category Module

1. Get All Categories

Method: GET

Endpoint: /Team3/Event/Routes/categories.php

Description: Retrieve all categories.

Authentication

Not explicitly configured.

Query Parameters

None.

Success Response

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "name": "Photography",
      "description": "Photography services"
    }
  ]
}
```

2. Get Category By ID

Method: GET

Endpoint: `/Team3/Event/Routes/categories.php?id=1`

Description: Retrieve a category by its ID.

Query Parameters

Name	Type	Required	Description
id	integer	Yes	Category ID

3. Add Category

Method: POST

Endpoint: `/Team3/Event/Routes/categories.php`

Description: Create a new category.

Headers

Content-Type: `application/json`

Request Body

```
{
  "Name": "Decor",
  "Description": "Decoration services"
}
```

4. Update Category

Method: PUT

Endpoint: `/Team3/Event/Routes/categories.php?id=9`

Description: Update an existing category.

Query Parameters

Name	Type	Required	Description
id	integer	Yes	Category ID

Request Body

```
{
  "Name": "Decor",
  "Description": "Updated description"
}
```

5. Delete Category

Method: DELETE

Endpoint: `/Team3/Event/Routes/categories.php?id=9`

Description: Delete a category by ID.

Query Parameters

Name	Type	Required	Description
id	integer	Yes	Category ID

Vendor Module

1. Get All Vendors

Method: GET
Endpoint: /Team3/Event/Routes/Vendor.php
Description: Retrieve all vendors.

2. Get Vendor By ID

Method: GET
Endpoint: /Team3/Event/Routes/Vendor.php?vendor_id=3
Description: Retrieve a single vendor by ID.

Query Parameters

Name	Type	Required	Description
vendor_id	integer	Yes	Vendor ID

3. Search Vendor By Name

Method: GET
Endpoint: /Team3/Event/Routes/Vendor.php?search=Bright Decor
Description: Search vendors by name.

Query Parameters

Name	Type	Required	Description
search	string	Yes	Vendor search term

4. Filter By Category

Method: GET
Endpoint: /Team3/Event/Routes/Vendor.php?categoryId=2
Description: Filter vendors by category.

Query Parameters

Name	Type	Required	Description
categoryId	integer	Yes	Category ID

5. Filter By Location

Method: GET
Endpoint: /Team3/Event/Routes/Vendor.php?location=cairo
Description: Filter vendors by location.

Query Parameters

Name	Type	Required	Description
location	string	Yes	City or location value

6. Combine Filters

Method: GET
 Endpoint: /Team3/Event/Routes/Vendor.php?categoryId=2&location=cairo
 Description: Filter vendors using multiple criteria.

Query Parameters

Name	Type	Required	Description
categoryId	integer	No	Category filter
location	string	No	Location filter

7. Filter by Day and Time

Method: GET
 Endpoint: /Team3/Event/Routes/Vendor.php?day=Friday
 Description: Filter vendors by availability day. The request name mentions time, but only day is currently saved.

Query Parameters

Name	Type	Required	Description
day	string	Yes	Day of availability

8. Get Active Vendors Only

Method: GET
 Endpoint: /Team3/Event/Routes/Vendor.php?action=active-vendors
 Description: Retrieve only active vendors.

9. Get Top Rated Vendors

Method: GET
 Endpoint: /Team3/Event/Routes/Vendor.php?action=top-rated
 Description: Retrieve top-rated vendors.

10. Update Vendor Data

Method: PUT
 Endpoint: /Team3/Event/Routes/Vendor.php?vendor_id=3
 Description: Update vendor profile data.

Request Body

```
{
  "name": "Bright Decor",
  "description": "Professional event decoration
    services",
  "categoryId": 2,
```

```
{
  "country": "Egypt",
  "governorate": "Cairo",
  "city": "Nasr City",
  "day": "Friday",
  "startTime": "10:00",
  "endTime": "18:00",
  "status": "active"
}
```

11. Activate Vendor

Method: PATCH
Endpoint: /Team3/Event/Routes/Vendor.php?vendor_id=3
Description: Activate or change vendor status.

Notes
The saved request contains ambiguous body data and should be reviewed before publishing production documentation.

12. Get Current Vendor

Method: GET
Endpoint: /3DOS/Event_Managment/Team3/Event/routes/Vendor.php/vendor/me
Description: Retrieve the authenticated vendor profile.

Authentication
Bearer token required.

Headers
Authorization: Bearer <token>

Package Module

1. Get All Packages
Method: GET
Endpoint: /test/Routes/Package.php?page=1&limit=10
Description: Retrieve a paginated list of packages.

Query Parameters			
Name	Type	Required	Description
page	integer	No	Page number
limit	integer	No	Number of records per page

2. Get Package By ID
Method: GET
Endpoint: /test/Routes/Package.php?id=1
Description: Retrieve a package by ID.

Query Parameters

Name	Type	Required	Description
id	integer	Yes	Package ID

3. Create Package

Method: POST
Endpoint: /test/Routes/Package.php
Description: Create a new package.

Headers

Content-Type: application/json

Request Body

```
{
  "vendorId": 1,
  "title": "Premium Wedding Package",
  "description": "Includes decoration and
    photography",
  "price": 5000,
  "activityStatus": "active"
}
```

4. Update Package

Method: PUT
Endpoint: /test/Routes/Package.php?id=1
Description: Update an existing package.

Query Parameters

Name	Type	Required	Description
id	integer	Yes	Package ID

Request Body

```
{
  "vendorId": 1,
  "title": "Updated Package Title",
  "description": "Updated package description",
  "price": 5500,
  "activityStatus": "active"
}
```

5. Delete Package

Method: DELETE
Endpoint: /test/Routes/Package.php?id=1
Description: Delete a package by ID.

6. Review Package

Method: POST
Endpoint: /test/Routes/Package.php?review=1&id=1
Description: Add a review to a package.

Query Parameters

Name	Type	Required	Description
review	integer	Yes	Review action flag
id	integer	Yes	Package ID

Request Body

```
{  
  "rating": 5,  
  "comment": "Excellent package and service"  
}
```

Order Module

1. Get Orders By Page

Method: GET
Endpoint: /task6/OrderRoute/OrderRoute.php/orders?page=1&limit=5
Description: Retrieve paginated orders.

Query Parameters

Name	Type	Required	Description
page	integer	No	Page number
limit	integer	No	Page size

2. Get Order By ID

Method: GET
Endpoint: /task6/OrderRoute/OrderRoute.php/orders?OrderID=2
Description: Retrieve a single order by ID.

Query Parameters

Name	Type	Required	Description
OrderID	integer	Yes	Order ID



3. Add Order

Method: POST

Endpoint: /task6/OrderRoute/OrderRoute.php/orders

Description: Create a new order.

Request Body

```
{
  "UserID": 1,
  "VendorID": 2,
  "StartTime": "2026-06-18 10:00:00",
  "EndTime": "2026-06-18 14:00:00"
}
```

4. Update Order

Method: PATCH

Endpoint: /task6/OrderRoute/OrderRoute.php/orders?OrderID=4

Description: Update an order, likely its status.

Query Parameters

Name	Type	Required	Description
OrderID	integer	Yes	Order ID

Request Body

```
{
  "status": "completed"
}
```

5. Delete Order

Method: DELETE

Endpoint: /task6/OrderRoute/OrderRoute.php/orders?OrderID=4

Description: Delete or cancel an order.

6. Wallet

Method: GET

Endpoint: /task6/OrderRoute/OrderRoute.php/wallet?vendorId=1